

```
# Install NLTK
```

```
!pip install nltk
```

```
import nltk
```

```
nltk.download('punkt')
```

```
nltk.download('stopwords')
```

```
nltk.download('wordnet')
```

```
nltk.download('averaged_perceptron_tagger_eng') #  FIXED
```

```
from nltk.tokenize import word_tokenize, sent_tokenize
```

```
sent = "I LOVE my India and it is the greatest country and that is a lie consider it"
```

```
print(sent_tokenize(sent))
```

```
print(word_tokenize(sent))
```

```
from nltk.corpus import stopwords
```

```
stop_words = set(stopwords.words("english"))
```

```
print(stop_words)
```

```
token = word_tokenize(sent)
```

```
cleaned_token = []
```

```
for w in token:
```

```
    if w not in stop_words:
```

```
        cleaned_token.append(w)
```

```
print("Tokenized Sentence:", token)
```

```
print("Filtered Sentence:", cleaned_token)
```

```
wr = [w.lower() for w in cleaned_token if w.isalpha()]
```

```
print(wr)
```

```

from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

port_stemmer_output = [stemmer.stem(w) for w in wr]
print(port_stemmer_output)


from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

lemmatizer_output = [lemmatizer.lemmatize(w) for w in wr]
print(lemmatizer_output)


# POS tagging
from nltk import pos_tag

token = word_tokenize(sent)
print(pos_tag(token))


cleaned_tokens = []

for w in token:
    if len(w.lower()) > 1:
        cleaned_tokens.append(w)


tagged = pos_tag(cleaned_tokens)
print(tagged)


# ===== TF-IDF =====


import pandas as pd

import math


documentA = "Jupiter is the largest Planet"
documentB = "Mars is the fourth planet from Sun"

```

```
bagOfWordsA = documentA.split(" ")
```

```
bagOfWordsB = documentB.split(" ")
```

```
uniqueWords = set(bagOfWordsA).union(set(bagOfWordsB))
```

```
numOfWordsA = dict.fromkeys(uniqueWords, 0)
```

```
for word in bagOfWordsA:
```

```
    numOfWordsA[word] += 1
```

```
numOfWordsB = dict.fromkeys(uniqueWords, 0)
```

```
for word in bagOfWordsB:
```

```
    numOfWordsB[word] += 1
```

```
def computeTF(wordDict, bagOfWords):
```

```
    tfDict = {}
```

```
    bagOfWordsCount = len(bagOfWords)
```

```
    for word, count in wordDict.items():
```

```
        tfDict[word] = count / float(bagOfWordsCount)
```

```
    return tfDict
```

```
tfA = computeTF(numOfWordsA, bagOfWordsA)
```

```
tfB = computeTF(numOfWordsB, bagOfWordsB)
```

```
def computeIDF(documents):
```

```
    N = len(documents)
```

```
    idfDict = dict.fromkeys(documents[0].keys(), 0)
```

```
    for document in documents:
```

```
        for word, val in document.items():
```

```
            if val > 0:
```

```
                idfDict[word] += 1
```

```
for word, val in idfDict.items():
    idfDict[word] = math.log(N / float(val))

return idfDict

ids = computeIDF([numOfWordsA, numOfWordsB])
print(ids)

def computeTFIDF(tfBagOfWords, ids):
    tfidf = {}
    for word, val in tfBagOfWords.items():
        tfidf[word] = val * ids[word]
    return tfidf

tfidfA = computeTFIDF(tfA, ids)
tfidfB = computeTFIDF(tfB, ids)

df = pd.DataFrame([tfidfA, tfidfB])
print(df)
```